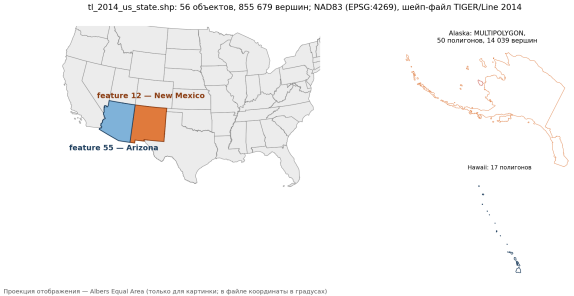
Векторные данные в Python

OGR, шейп-файлы, разбор геометрии и расстояние по большому кругу



Шейп-файл `tl_2014_us_state.shp`: 56 объектов, 855 679 вершин, NAD83 (EPSG:4269)

Разбор материалов главы 2

Репозиторий `Python-Geospatial-Development-Third-Edition`
папка `Chapter 02`: `example01.py` … `example05.py`
Продолжительность занятия: 90 минут

План занятия (90 минут)

Часть I. Данные и формат

Время	Тема
5 мин	Что лежит в папке `Chapter 02`
5 мин	Шейп-файл: пять файлов, бинарный формат
4 мин	СК, атрибуты, объём данных

Часть II. Чтение через OGR

6 мин	Объектная модель OGR
8 мин	`example01`: слои, СК, объекты
7 мин	`example02`: атрибуты и геометрия
8 мин	`example03`: рекурсивный разбор
7 мин	`example04`: крайние точки, производительность

Часть III. Геодезия

Время	Тема
2 мин	Зачем нам расстояние
9 мин	`example05`: формула гаверсинуса
6 мин	Геометрия формулы и её вывод
7 мин	Проверка на реальных данных

Итоги занятия

- объектная модель OGR и рекурсия по геометрии;
- устройство шейп-файла «изнутри»;
- геодезия на сфере и её границы.

Что лежит в папке `Chapter 02`

В папке пять коротких программ и *ни одного файла данных*.

Файл	Что показывает
`example01.py`	Обход источника данных: слои, система координат, количество объектов и их имена
`example02.py`	Атрибуты одного объекта и имя его геометрии
`example03.py`	Рекурсивный «разбор» геометрии: дерево типов и числа точек
`example04.py`	Рекурсивный поиск самой северной и южной вершины объекта
`example05.py`	Расстояние между двумя точками по большому кругу (формула гаверсинуса)

Данные, которые нужны примерам

Шейп-файл `tl_2014_us_state.shp` — границы штатов США из набора *TIGER/Line 2014* (US Census Bureau).

Все числа в этой презентации получены запуском этих пяти файлов именно на этом шейп-файле.

Все примеры запускаются «как есть», если рядом с ними лежит шейп-файл `tl_2014_us_state.shp`.

Главная мысль

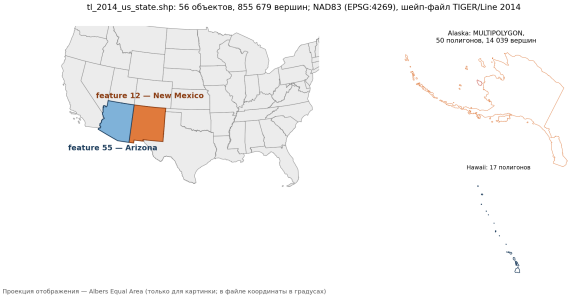
Пять примеров образуют один сюжет: **открыть файл** → **понять, что внутри** → **рекурсивно обойти геометрию** → **посчитать по ней географию**.

Часть I. Данные и формат

Шейп-файл: что это за файл и как он устроен

25 минут

Данные: границы 56 единиц США



На карте выделены два объекта, с которыми работают примеры: **feature 12 — New Mexico** и **feature 55 — Arizona**. Верхний правый фрагмент иллюстрирует первую проблему главы: Alaska — это *MULTIPOLYGON* из 50 отдельных полигонов, а Hawaii — из 17.

Шейп-файл = не один файл, а пять

Таблица: Состав набора на диске (реальные размеры)

Файл	Размер	Назначение
`.shp`	13 694 480	геометрии
`.shx`	548	индекс записей
`.dbf`	10 450	атрибуты
`.prj`	165	система координат (WKT)
`...`		также `.cpg`, `.qpj`

- **геометрия и атрибуты разнесены** по разным файлам;
- `.shx` — индекс: по номеру объекта сразу находится его смещение в `.shp`;
- это главный аргумент «за» OGR: формат разбирает библиотека, а не вы.

А если файла нет?

`example01.py` вызовет `ogr.Open("tl_2014_us_state.shp")` и упадёт с `NULL` в переменной `shapefile`.

Проверка, которую стоит писать в каждом проекте:

```
from osgeo import ogr, gdal
gdal.UseExceptions()      # GDAL >= 3.7

ds = ogr.Open("tl_2014_us_state.shp")
if ds is None:
    raise IOError("файл не найден")
```

Что внутри `.shp`: заголовок и записи

Таблица: Заголовок `.shp` — первые 100 байт

Смещение	Значение в нашем файле
0–3	File code = 9994 (главный файл)
24–27	Длина файла: 6 847 240 16-битных слов
28–31	Version = 1000
32–35	Shape type = **5** (PolygonZ)
36–99	Extent: $-179.231086,\ 179.859681,$ -14.601813, 71.441059

Дальше — **56 записей**: номер записи (8 байт), длина содержимого (8 байт) и координаты, по 16 байт на точку (тип `double`).

Таблица: `.shx`: $100 + 56 \times 8 = 548$ байт

Запись	Смещение и длина
№ 0	50 слов = 100 байт; 281 280 слов = 562 560 байт
№ 55	6 768 412 слов = 13 536 824 байта

Как читается `GetFeature(12)`

Из `.shx` берутся 8 байт с позиции $100 + 12 \times 8$. Первое число, умноженное на 2, — смещение записи в `.shp`; второе — длина её содержимого.

Вывод: порядок объектов фиксирован и *не* алфавитный.

Что лежит в атрибутах: 14 полей DBF

Таблица: Схема слоя `tl_2014_us_state` (полный состав полей)

Поле	Тип	Поле	Тип	Поле	Тип
REGION	String(2)	GEOID	String(2)	ALAND	Integer64
DIVISION	String(2)	STUSPS	String(2)	AWATER	Integer64
STATEFP	String(2)	NAME	String(100)	INTPTLAT	String(11)
STATENS	String(8)	LSAD	String(2)	INTPTLON	String(12)
MTFCC	String(5)	FUNCSTAT	String(1)		

- **STATEFP / GEOID** — двухзначный код штата по FIPS;
- **NAME** — название: `"New Mexico"`;
- **ALAND / AWATER** — площадь суши и воды в м^2: у Нью-Мэксико $314\,161\,410\,324$ м^2 суши;
- **INTPTLAT / INTPTLON** — «внутренняя точка», хранится *строкой*, потому что содержит точку и знак.

Подводный камень

В DBF нет типов «строка с точкой» и «число с плавающей точкой»: `ALAND` — это `Integer64`, а `INTPTLAT` — `String`. Преобразовывать придётся вручную: `float(f.GetField("INTPTLAT"))`.

Система координат: NAD83 (EPSG:4269)

Что сообщает программа

```
Layer 0 has spatial reference
+proj=longlat +datum=NAD83 +no_defs
```

- Координаты в файле — **градусы**, а *не* метры;
- `longlat` — географическая (угловая) СК;
- `datum=NAD83` — эллипсоид и система отсчёта: **North American Datum 1983**, код **EPSG:4269**;
- `+no_defs` — «дополнительных определений нет».

Таблица: Что ещё лежит в объекте СК

Вызов	Результат
`GetName()`	NAD83
`GetAuthorityName`	EPSG
`GetAuthorityCode`	4269
`ExportToProj4()`	строка PROJ.4
`ExportToWkt()`	полное WKT-описание

Зачем это знать

Расстояние в градусах — не метры. Отсюда растёт вся часть III: чтобы получить **километры**, формула должна знать радиус Земли.

Объектная модель OGR



число слоёв: `GetLayerCount()` пространственный фильтр (опц.)

имя слоя, СК, число объектов, границы, предел `SetFeature()`

поля атрибутов, FID, геометрия

тип, число точек, число потомков, точки по индексу

Один и тот же объект `Layer` умеет и читать, и фильтровать, и считать — а `Geometry` умеет разбирать сам себя рекурсивно.

Иерархия геометрий: от контейнера к точке

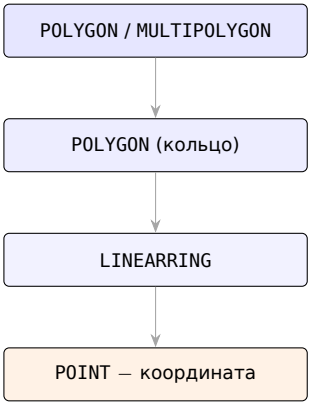


Таблица: Два метода — это и есть рекурсия

Вызов	Что возвращает
`GetGeometryCount()`	число непосредственных *детей*
`GetGeometryRef(i)`	*i*-й ребёнок
`GetPointCount()`	число точек *внутри* узла
`GetPoint(i)`	координаты *i*-й точки

Ловушка, на которой спотыкаются все

Для `POLYGON` метод `GetPointCount()` возвращает **0**: точки лежат не в самом полигоне, а в его кольцах. Поэтому «просто пройтись по точкам» не работает — нужен спуск по уровням.

Часть II. Чтение данных через OGR

Пять программ — пять шагов вглубь

55 минут

example01.py: открыть и перечислить

```
import osgeo.ogr

shapefile = osgeo.ogr.Open("tl_2014_us_state.shp")
numLayers = shapefile.GetLayerCount()

print("Shapefile contains {} layers".format(numLayers))
print()

for layerNum in range(numLayers):
    layer = shapefile.GetLayer(layerNum)
    spatialRef = layer.GetSpatialRef().ExportToProj4()
    numFeatures = layer.GetFeatureCount()
    print("Layer {} has spatial reference {}".format(layerNum, spatialRef))
    print("Layer {} has {} features".format(layerNum, numFeatures))
    print()

    for featureNum in range(numFeatures):
        feature = layer.GetFeature(featureNum)
        featureName = feature.GetField("NAME")
        print("Feature {} has name {}".format(featureNum, featureName))
```

Что эта программа выводит на наших данных

```
Shapefile contains 1 layers

Layer 0 has spatial reference +proj=longlat +datum=NAD83 +no_defs
Layer 0 has 56 features

Feature 0 has name West Virginia
Feature 1 has name Florida
Feature 2 has name Illinois
Feature 3 has name Minnesota
Feature 4 has name Maryland
Feature 5 has name Rhode Island
Feature 6 has name Idaho
Feature 7 has name New Hampshire
Feature 8 has name North Carolina
Feature 9 has name Vermont
Feature 10 has name Connecticut
Feature 11 has name Delaware
Feature 12 has name New Mexico
Feature 13 has name California
Feature 14 has name New Jersey
Feature 15 has name Wisconsin
   ...
Feature 55 has name Arizona
```

Три наблюдения: **(1)** в наборе **56** объектов — 50 штатов, DC, Puerto Rico, Guam, American Samoa, Northern Mariana Islands и US Virgin Islands; **(2)** порядок *не* алфавитный; **(3)** `feature 12` — это **New Mexico**, `feature 55` — **Arizona**.

Разбор по строкам: что здесь важно

Таблица: Разбор ключевых строк example01.py

Строка	Что происходит
`ogr.Open("...shp")`	Возвращает DataSource. Чтение ленивое: файл открыт, но данные ещё не прочитаны.
`GetLayerCount()`	У шейп-файла всегда **один** слой с именем `tl_2014_us_state`.
`GetLayer(0)`	Слой — это «таблица»: строки-объекты и общая СК с геометрией.
`layer.GetSpatialRef()`	СК слоя. Значение None возможно, только если СК не задана; здесь она задана.
`GetFeatureCount()`	**56**. Это число, а не признак «данные уже в памяти».
`GetFeature(12)`	Читает 13-ю запись: атрибуты *и* ссылку на геометрию.
`GetField("NAME")`	Достаёт поле по имени. В DBF имена чувствительны к регистру.

Три способа пройти по объектам: по индексу (`GetFeature(i)`), «как по словарю» (`for f in layer`) или SQL-фильтром (`GetLayerByName(...).ExecuteSQL(...)`).

Порядок объектов: FIPS, а не алфавит

Таблица: Первые объекты файла и их коды FIPS

№	NAME	STATEFP
0	West Virginia	54
1	Florida	12
2	Illinois	17
3	Minnesota	27
4	Maryland	24
5	Rhode Island	44
6	Idaho	16
…	…	…
55	Arizona	04

Ни алфавит, ни сортировка по FIPS: это порядок, зафиксированный при сборке набора.

Практический вывод

- «Нью-Мэксико — это 12-й объект» верно только *для этого файла*;
- чтобы найти нужный штат, ищите по полю STATEFP / STUSPS, а не по номеру;
- в NAME есть и `Commonwealth of the Northern Mariana Islands` — сравнивать строки надо аккуратно.

```
lyr.SetAttributeFilter("NAME = 'Arizona'")
for f in lyr:
    print(f.GetFID())
```

example02.py: атрибуты одного объекта

```
1  import osgeo.ogr
2
3  shapefile = osgeo.ogr.Open("tl_2014_us_state.shp")
4  layer = shapefile.GetLayer(0)
5  feature = layer.GetFeature(12)
6
7  print("Feature 12 has the following attributes:")
8  print()
9
10 attributes = feature.items()
11
12 for key,value in attributes.items():
13     print(" {} = {}".format(key, value))
14
15 geometry = feature.GetGeometryRef()
16 geometryName = geometry.GetGeometryName()
17
18 print()
19 print("Feature's geometry data consists of a {}".format(geometryName))
```

Вывод: 14 полей и одна геометрия

```
Feature 12 has the following attributes:

 REGION = 4
 DIVISION = 8
 STATEFP = 35
 STATENS = 00897535
 GEOID = 35
 STUSPS = NM
 NAME = New Mexico
 LSAD = 00
 MTFCC = G4000
 FUNCSTAT = A
 ALAND = 314161410324
 AWATER = 755712514
 INTPTLAT = +34.4346843
 INTPTLON = -106.1316181

Feature's geometry data consists of a POLYGON
```

`items()` — «словарь всех полей»; порядок полей в нём совпадает с порядком полей в DBF, то есть с порядком объявления схемы, а не с алфавитным.

Три способа достать атрибуты

Таблица: API для атрибутов

Вызов	Результат
`GetField("NAME")`	`"New Mexico"`
`GetField(6)`	`"New Mexico"`
`GetFieldAsString(6)`	строка в любом случае
`GetFieldAsInteger(6)`	целое, если поле числовое
`items()`	словарь всех полей
`keys()`	список имён полей

```
1  geom = f.GetGeometryRef()
2  print(geom.GetGeometryName())
3  print(geom.GetEnvelope())
4  print(geom.Centroid().ExportToWkt())
```

`GetGeometryRef()` возвращает *ссылку*, а не копию: присваивание `g = f.GetGeometryRef()` не удваивает память. Для Нью-Мэксико `GetEnvelope()` даёт `(-109.050173, -103.001964, 31.332172, 37.000293)` — то есть `(xmin, xmax, ymin, ymax)` в градусах.

example03.py: рекурсивный разбор геометрии

```
1  import osgeo.ogr
2
3  def analyzeGeometry(geometry, indent=0):
4      s = []
5      s.append("  " * indent)
6      s.append(geometry.GetGeometryName())
7      if geometry.GetPointCount() > 0:
8          s.append(" with %d data points" % geometry.GetPointCount())
9      if geometry.GetGeometryCount() > 0:
10         s.append(" containing:")
11
12     print("".join(s))
13
14     for i in range(geometry.GetGeometryCount()):
15         analyzeGeometry(geometry.GetGeometryRef(i), indent+1)
16
17 shapefile = osgeo.ogr.Open("tl_2014_us_state.shp")
18 layer = shapefile.GetLayer(0)
19 feature = layer.GetFeature(12)
20 geometry = feature.GetGeometryRef()
21
22 analyzeGeometry(geometry)
```

Вывод: два уровня вложенности

```
POLYGON containing:
  LINEARRING with 7554 data points
```

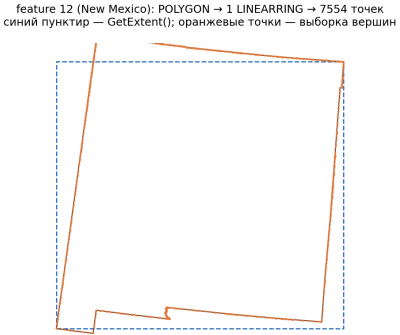
Отступ `"  " * indent` — это и есть «дерево», напечатанное текстом. Разбор занял **19 строк** кода, и это **единственный** алгоритм, который одинаково работает и для одного кольца, и для Аляски с 50 островами.

Почему рекурсия

Структура OGR — это *дерево*, а глубина зависит от типа объекта. Фиксированное число уровней здесь — ловушка: у `MULTIPOLYGON` их четыре.

```
MULTIPOLYGON containing:   # Alaska
  POLYGON containing:
    LINEARRING with 180 data points
  POLYGON containing:
    LINEARRING with 86 data points
  ... ещё 48 полигонов
```

Кольцо из 7554 точек



Нью-Мэксико — почти идеальный прямоугольник в географических координатах: границы идут по линиям $\varphi = \text{const}$ и $\lambda = \text{const}$. Синим пунктиром — результат `GetExtent()`, оранжевые точки — выборка вершин из `GetPoint(i)`.

Сколько колец у разных объектов

Таблица: 12 объектов из 56 содержат больше одного кольца

Объект	Колец	Вершин	Объект	Колец	Вершин
Alaska	**50**	14 039	American Samoa	4	510
Hawaii	**17**	3 523	New Jersey	3	6 140
Commonwealth of the N. Marian Is.	**12**	818	New York	3	9 976
California	7	11 131	Puerto Rico	3	1 565
Florida	2	19 798	Kentucky	2	20 306
Rhode Island	2	2 541	U.S. Virgin Is.	2	495

- **Texas** — рекордсмен по числу вершин: **58 455**;
- минимум — **US Virgin Islands**: **495**;
- **всего по набору** — **855 679** вершин.

Мораль

Код, который «знает», что у штата одно кольцо, сломается на Аляске. Рекурсия из `example03` написана так, чтобы работать с любым случаем — это главный приём главы.

example04.py: рекурсивный поиск крайних точек

```
1  import osgeo.ogr
2
3  def findPoints(geometry, results):
4      for i in range(geometry.GetPointCount()):
5          x,y,z = geometry.GetPoint(i)
6          if results['north'] == None or results['north'][1] < y:
7              results['north'] = (x,y)
8          if results['south'] == None or results['south'][1] > y:
9              results['south'] = (x,y)
10
11     for i in range(geometry.GetGeometryCount()):
12         findPoints(geometry.GetGeometryRef(i), results)
13
14 shapefile = osgeo.ogr.Open("tl_2014_us_state.shp")
15 layer = shapefile.GetLayer(0)
16 feature = layer.GetFeature(55)
17 geometry = feature.GetGeometryRef()
18
19 results = {'north' : None,
20            'south' : None}
21
22 findPoints(geometry, results)
23
24 print("Northernmost point is ({:.4f}, {:.4f})".format(
25       results['north'][0], results['north'][1]))
26 print("Southernmost point is ({:.4f}, {:.4f})".format(
27       results['south'][0], results['south'][1]))
```

Вывод: Arizona, 9850 вершин

```
Northernmost point is (-110.5007, 37.0043)
Southernmost point is (-111.0006, 31.3322)
```



Это **feature 55** — Arizona. Одна функция — два результата.

Красный отрезок — путь между найденными крайними вершинами; он же совпадает с границами `GetEnvelope()` по вертикали: 37.0043 и 31.3322.

Что здесь сделано хорошо и что можно улучшить

Хорошо

- **одна функция — два результата**: словарь `results` играет роль «накопителя»;
- **два независимых прохода**: сначала все точки текущего узла, потом рекурсия по детям;
- **сравнение через `None`**: алгоритм работает и с первого вызова.

Что улучшить

- `== None` лучше писать как `is None`;
- есть только «север» и «юг» — запад и восток добавляются так же (сравнение по *x*);
- «север» и «юг» условны: речь о максимуме и минимуме широты *в градусах*;
- кольцо замкнуто и идёт по границе, поэтому экстремум по *y* можно искать вообще без Python-цикла.

Тот же результат силами OGR

`geometry.GetEnvelope()` или `layer.GetExtent()` — без единой вершины в Python-коде.

```
1  g = lyr.GetFeature(55).GetGeometryRef()
2  xmin, xmax, ymin, ymax = g.GetEnvelope()
3  print(f"{ymin:.4f} {ymax:.4f}")
4  # 31.3322 37.0043  <- совпало
```

Производительность: во что обходится обход

Измерено на нашем файле (Python 3, GDAL 3.13, один поток):

Таблица: Замеры на feature 55 (Arizona, 9850 вершин)

Операция	Время	Комментарий
`ogr.Open` + `GetLayerCount`	~0 мс	файл открыт, но не прочитан
`GetFeature(55)` (первое обращение)	11.7 мс	читается запись из `.shp`
`GetPoint(i)` в цикле по Python	5.0 мс	9850 вызовов через SWIG
`GetPoints()` целиком (OGR)	0.9 мс	**в 5.6 раза быстрее**
`GetEnvelope()`	0.02 мс	**в 250 раз быстрее**
рекурсивный обход всех 56 объектов	15 мс	все **855 679** вершин

Правило

Простой `GetPoint(i)` удобен для обучения и точечных запросов. Для массовых операций спрашивайте у OGR сразу всё (`GetPoints()`, `GetEnvelope()`, `ExportToWkb()`), а не перебирайте вершины по одной.

Подводные камни, о которые стоит споткнуться

1. **`GetPointCount()` у POLYGON = 0.** Точки лежат в кольцах. Проверили: `(0, 1, LINEARRING, 7554)`.
2. **Третья координата.** `GetPoint(i)` возвращает `(x, y, z)`, а в нашем файле $z = 0$. Распаковка `x, y, z = ...` работает, `x, y = ...` — нет.
3. **Нет обработки ошибок.** `ogr.Open` вернёт `None`, и через три строки вы получите `AttributeError` вместо внятного сообщения.
4. **Устаревший API.** Современный GDAL (3.7+) по умолчанию бросает исключения вместо `None` — включите `gdal.UseExceptions()` явно.

И ещё

- **порядок** объектов не гарантирован ничем;
- **кольцо замкнуто**: первая и последняя точки совпадают (проверили: `closed=True`);
- **антимеридиан**: extent набора уходит в $\lambda \in [-179.23;\ 179.86]$ — Аляска «перескакивает» на $+180°$, и наивная проекция даст артефакты;
- **доминантное кольцо**: в многоугольнике с «дырами» первое кольцо — внешнее, остальные — внутренние.

```
r = g.GetGeometryRef(0)
n = r.GetPointCount()
print(r.GetPoint(0) == r.GetPoint(n-1))
# True
```

Часть III. Геодезия

Откуда в векторной главе вдруг появилась тригонометрия

30 минут

Зачем вдруг расстояние между точками?

- Данные из шейп-файла лежат в **градусах**. Чтобы сравнить «длину границы» с «площадью» или ответить на вопрос «насколько далеко?», градусы надо превратить в метры.
- Геодезия решает это на **эллипсоиде** — но сложно.
- Простейшая математическая модель Земли — **сфера**, одна константа: радиус R.
- Путь по поверхности сферы, соединяющий две точки, — дуга **большого круга**. Она кратчайшая.
- Значит, задача сводится к поиску длины этой дуги.

Две координаты из примера

	широта	долгота
точка 1	42.0095	-122.3782
точка 2	32.5288	-117.2049

Это [*Crater Lake*, Орегон] и [*San Diego*, Калифорния]. Наклон $\Delta\varphi = -9.48^\circ$, разброс по долготе $\Delta\lambda = +5.17^\circ$.

example05.py: гаверсинус в 20 строк

```
import math

lat1 = 42.0095
long1 = -122.3782

lat2 = 32.5288
long2 = -117.2049

rLat1 = math.radians(lat1)
rLong1 = math.radians(long1)

rLat2 = math.radians(lat2)
rLong2 = math.radians(long2)

dLat = rLat2 - rLat1
dLong = rLong2 - rLong1
a = math.sin(dLat/2)**2 + math.cos(rLat1) * math.cos(rLat2) \
                        * math.sin(dLong/2)**2
c = 2 * math.atan2(math.sqrt(a), math.sqrt(1-a))
distance = 6371 * c

print("Great circle distance is {:.0f} kilometers".format(distance))
```

Разбор формулы по шагам

Таблица: Что происходит в каждой строке

Строка	Смысл
`math.radians(...)`	градусы $\to$ радианы: тригонометрия Python работает только в радианах
`dLat, dLong`	разности широт и долгот в радианах: -0.165469 и 0.090291
`math.sin(dLat/2)**2`	$\mathrm{hav}\,\Delta\varphi$ — «полухорд»
`math.cos(rLat1)*…`	$\cos\varphi_1 \cos\varphi_2$ — поправка на широту
`a`	значение подкоренного выражения: **0.0081053792**
`atan2(…)`	устойчивый «обратный синус»: не теряет точность при $a \approx 1$
`6371 * c`	средний радиус Земли $\times$ центральный угол

Зачем atan2, а не asin

Наивный вариант `c = 2*math.asin(math.sqrt(a))` ломается, когда точки близки или совпадают: $\sqrt{a} \to 1$, производная `asin` уходит в бесконечность. `atan2` считает ту же величину устойчиво.

Про радиус

6371 км — средний радиус по Объединённому геодезическому эллипсоиду. От его выбора напрямую зависит точность.

Формула целиком

$$\operatorname{hav}\sigma = \sin^2\frac{\sigma}{2} = \operatorname{hav}\Delta\varphi + \cos\varphi_1 \cdot \cos\varphi_2 \cdot \operatorname{hav}\Delta\lambda$$

$$\sigma = 2\operatorname{atan2}\left(\sqrt{a},\ \sqrt{1-a}\right), \qquad d = R\,\sigma$$

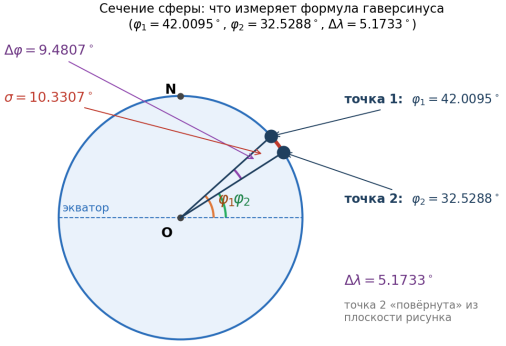
- $\operatorname{hav}(\theta) \equiv \sin^2(\theta/2)$ — «полухорд»;
- σ — **центральный угол** между точками (угол в центре сферы);
- a — значение в скобках; оно всегда в $[0;\,1]$, поэтому оба корня вещественны;
- d — длина дуги: **радиусы** умножаются на **угол в радианах**.

Почему $\cos\varphi_1 \cos\varphi_2$

Разница долгот $\Delta\lambda$ сама по себе не даёт расстояния: на экваторе 1° долготы ≈ 111 км, а на 60° с.ш. — всего 55 км. Множитель $\cos\varphi_1 \cos\varphi_2$ это учитывает.

На наших данных:
$\cos 42.0095^\circ \cdot \cos 32.5288^\circ = 0.626468$.

Что означает формула геометрически



Три «числа» в формуле — это стороны прямоугольного сферического треугольника: $\Delta\varphi$ (разница широт), $\Delta\lambda$ (разница меридианов) и σ (центральный угол между точками).

Откуда берётся формула: тригонометрия в 4 строки

На сфере радиуса R длины дуг пропорциональны углам, поэтому достаточно работать с *углами* треугольника со сторонами $\left(\frac{\pi}{2} - \varphi_1\right)$, $\left(\frac{\pi}{2} - \varphi_2\right)$ и σ.

1. **Сферическая теорема косинусов** для прямого угла при A:

$$\cos\sigma = \sin\varphi_1 \sin\varphi_2 + \cos\varphi_1 \cos\varphi_2 \cos\Delta\lambda.$$

2. Подставляем $\cos\Delta\lambda = 1 - 2\,\mathrm{hav}\,\Delta\lambda$ и $\cos\sigma = 1 - 2\,\mathrm{hav}\,\sigma$:

$$1 - 2\,\mathrm{hav}\,\sigma = 1 - 2\,\mathrm{hav}\,\Delta\varphi - 2\cos\varphi_1 \cos\varphi_2\,\mathrm{hav}\,\Delta\lambda.$$

3. Сокращаем единицы и делим на 2:

$$\mathrm{hav}\,\sigma = \mathrm{hav}\,\Delta\varphi + \cos\varphi_1 \cos\varphi_2\,\mathrm{hav}\,\Delta\lambda.$$

4. Берём $\sigma = 2\,\mathrm{atan2}(\sqrt{a}, \sqrt{1-a})$ и умножаем на R.

Откуда взялся именно «полухорд»

Тождества $\cos\Delta\lambda = 1 - 2\sin^2\frac{\Delta\lambda}{2}$ и $\cos\sigma = 1 - 2\sin^2\frac{\sigma}{2}$ — это формулы половинного угла. Именно они убирают из формулы *радианные разности* $\Delta\varphi$, $\Delta\lambda$, которые иначе пришлось бы считать через `radians`. Это делает формулу устойчивой: hav всегда остаётся маленьким числом.

Числа примера по шагам

Таблица: Промежуточные значения (вычислены независимо, совпадают с программой)

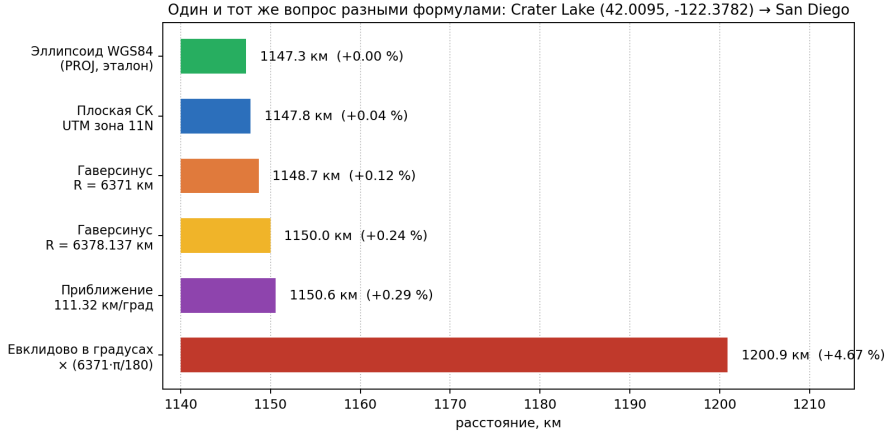
Величина		Значение	Величина		Значение
$\Delta\varphi$	$=$	-0.165469 рад	$\cos\varphi_1 \cos\varphi_2$	$=$	0.626468
$\Delta\varphi$	$=$	$-9.4807°$	$\operatorname{hav}\Delta\varphi$	$=$	0.0068294
$\Delta\lambda$	$=$	0.090291 рад	$\operatorname{hav}\Delta\lambda$	$=$	0.0020367
$\Delta\lambda$	$=$	$5.1733°$	a	$=$	0.0081053792
σ	$=$	0.1803039 рад	σ	$=$	$10.3307°$
R	$=$	6371 км	$d = R\sigma$	$=$	**1148.716 км**

Программа печатает с форматом `{:.0f}` — то есть **1149** kilometers. Округление до целого скрывает всю точность формулы: полезно знать и то, и другое.

Для самопроверки

Подставьте в код $R = 6378.137$: получится **1150.003** км. Разница **1.29** км целиком объясняется выбором радиуса.

Проверка на настоящей геодезии



Эталон — решение на эллипсоиде WGS84 (`pyproj`, метод геодезических линий): **1147.3** км. Гаверсинус со средним радиусом отличается на **+0.12 %** — это примерно 1.4 км на дистанции в 1150 км, то есть на дальних расстояниях приемлемо.

Когда какой метод уместен

Таблица: Сравнение подходов на нашем примере

Метод	Результат	Ошибка
Геодезия на эллипсоиде (PROJ/WGS84)	1147.3 км	— (эталон)
Плоская СК UTM, зона 11N	1147.8 км	$+0.04\,\%$
Гаверсинус, $R = 6371$	**1148.7 км**	$+0.12\,\%$
Гаверсинус, $R = 6378.137$	1150.0 км	$+0.24\,\%$
«111.32 км на градус» с $\cos\varphi$	1150.6 км	$+0.29\,\%$
Евклидово в градусах $\times(6371\pi/180)$	1200.9 км	$+4.67\,\%$

Берите

- **гаверсинус** — почти всегда достаточно: быстро и без внешних зависимостей, точность $\sim 0.5\,\%$ на любой дистанции;
- **UTM** — когда нужны метры в плоскости;
- **геодезию PROJ** — когда важны сантиметры.

Осторожно

- последний способ — ловушка: «градусы $\times$ 111» даёт ошибку $4.7\,\%$;
- при atan2 не забывайте про **радианы**;
- UTM-зона — *полоса*, а не глобальная проекция.

Чек-лист «как не ошибиться»

1. **Градусы вместо радиан.** `math.sin(9.48)` вместо `math.radians(9.48)` — тихая ошибка на 3–4 порядка.
2. **Перепутаны широта и долгота.** В PROJ и OGR порядок **(x, y) = (долгота, широта)**; в `GetPoint` — тоже. В списке людей — наоборот.
3. **Радиус в километрах, результат в метрах** (или наоборот).
4. **Нет проверки на `None`** после `ogr.Open` и `GetFeature`.
5. **Забыли про рекурсию**: у `POLYGON` точки напрямую не читаются.

```
# метры -> километры явно
d_km = 6371.0 * c

# проверка "на глаз"
assert 0.0 <= a <= 1.0
assert d_km > 0

# безопасное чтение
ds = ogr.Open(path)
if ds is None:
    raise IOError(path)
f = ds.GetLayer(0).GetFeature(55)
if f is None:
    raise KeyError("нет объекта 55")
```

Хорошая привычка

Перед расчётом расстояния напечатайте `ExportToProj4()`. Если там не градусы — формула гаверсинуса бессмысленна.

Куда движется глава 3

Что мы сделали «руками»

- чтение шейп-файла — `osgeo.ogr` (OGR, C++);
- разбор геометрии — те же рекурсивные методы OGR;
- расстояние — своя формула на `math`.

Это и делают инструменты главы 3

- `Fiona` — векторные данные (обёртка над OGR);
- `Pyproj` — СК и геодезические расчёты;
- `Shapely` — геометрия и операции над ней;
- `GDAL` — растры.

```
1  # example05, но на эллипсоиде
2  from pyproj import Geod
3
4  g = Geod(ellps="WGS84")
5  _, _, d = g.inv(-122.3782, 42.0095,
6                  -117.2049, 32.5288)
7  print(d / 1000)
8  # 1147.303...  <- точнее
9
10 # и на Shapely: та же сфера
11 from shapely.geometry import Point
12 p1 = Point(-122.3782, 42.0095)
13 p2 = Point(-117.2049, 32.5288)
14 print(p1.distance(p2) / 1000)
15 # 1148.7...
```

То есть та же математика, только реализованная один раз и со всеми проверками.

Итоги: пять тезисов

1. **Шейп-файл** — это пять файлов (геометрия, индекс, атрибуты, CK), и порядок объектов в нём произвольный: ищите объекты по полям, а не по номерам.
2. **OGR** даёт стройную объектную модель `DataSource` $\rightarrow$ `Layer` $\rightarrow$ `Feature` $\rightarrow$ `Geometry`; все четыре уровня нужны для простой задачи.
3. **Геометрия — это дерево**, а не список точек: рекурсия по `GetGeometryCount()`/`GetGeometryRef()` обязательна, а `GetPointCount()` у `POLYGON` равен нулю.
4. **Рекурсия решает и «бытовые» задачи**: крайние точки штата находятся за 20 строк — а `GetEnvelope()` даёт то же самое за одну.
5. **Гаверсинус** — переход от градусов к километрам через сферу: ошибка $\sim 0.5\,\%$ вместо $4.7\,\%$ у «градусных» формул. Для сантиметров нужен эллипсоид PROJ.

Ключевая мысль главы

Геопространственные данные — это *структура*, а не файл. Пока вы понимаете, как устроен объект изнутри (поля + дерево геометрии + CK), любой формат читается одинаково.

Практика

1. Запустить всё. Положить рядом `tl_2014_us_state.shp` и выполнить пять примеров. Сверить вывод с числами этой презентации (56 объектов, feature 12 = New Mexico, feature 55 = Arizona, 1149 км).

2. Найти, а не угадать. Модифицировать `example01.py` так, чтобы он печатал только объекты с `ALAND` > 1 000 000 000 м^2 (10 штатов-гигантов), и проверить, что среди них есть Texas.

3. Рекурсия. Написать функцию, которая для заданного номера объекта печатает его *тип* и *число колец*. Прогнать по всем 56 объектам: 44 объекта с одним кольцом и 12 — с несколькими.

4. Экстремумы. Дополнить `example04.py` поиском самых западной и восточной вершин и сравнить результат с `GetEnvelope()`.

5. Геодезия. Сравнить три реализации расстояния между теми же двумя точками: `example05.py`, `pyproj.Geod` и `Shapely` (`Point.distance`). Построить таблицу отклонений; проверить, что при `R = 6378.137` гаверсинус даёт 1150.0 км, а на эллипсоиде — 1147.3 км.

6. *(дополнительно)* Посчитать площадь Нью-Мэксико двумя способами: `geometry.GetArea()` в квадратных градусах и «через UTM» в квадратных километрах, затем сравнить с `ALAND` из атрибутов.

Вопросы?

Пять файлов, один шейп-файл, **855 679** вершин

`example01.py`	слои, СК, объекты
`example02.py`	атрибуты и имя геометрии
`example03.py`	дерево геометрии
`example04.py`	крайние точки
`example05.py`	расстояние по большому кругу